UC San Diego

Self-Evolving Game Agent with MCTS and Tool-Integration

CSE 291A - AI Agents - Team 9

Jason Zhu, Yinuo Zhou, Hongrui (Frank) Zhang, Yaran Yang, Zavier Zhao, Fengfei Yu

University of California, San Diego | Mar 12th, 2026

Introduction

Motivation & Core Problems

- Solving turn-based games as MDP problems has been an effective strategy (Go, etc.)
- One method is **Monte Carlo Tree Search**. it searches the game environment and finds promising paths algorithmically.
- AlphaZero style MCTS **trains deep neural networks** for state value estimations, which is hard to setup right and has massive entry cost.
- Earlier MCTS+LLM work focused on **online, step-wise LLM** guidances, which is costly and constrained by static local views.
- Emergence of LLM skills/mcp gave us motivation for another direction: **policy improvement with rule based heuristics proposed autonomously by LLM.**

Introduction

- We developed a **LLM tool-evolving agent** for **turn-based games:**
  - Interacts with different game environments (Sokoban, TextWorld, etc.)
  - Collect game traces, analyze feedback, do **task proposal**, generate pluggable heuristics, improve upon successful validation. Iterate.
  - Mainly used **Monte Carlo Tree Search** as a foundational framework, but also explored other ideas like **A*** for games our MCTS struggled.
  - **Generates heuristics** for MCTS, saw improvement even with low iterations.
  - LLM acts as an **offline coding agent**, while code based heuristics run online efficiently. Contrast with using LLM as an online decision agent or training deep neural networks.
  - Generated heuristics are **human understandable**, code artifacts instead of weights.

Audience of Interest

- **LLM users and researchers:**
  - We explore the application of **LLM tool creation**, a forefront research direction
  - We explore **self-evolving** agent paradigm, populating and optimizing a tool collection
  - **Task proposal** - we explored LLM's ability to learn from environment feedback and propose improvements autonomously, alike human players' improvement from experience.
- **Game players:**
  - Generating heuristics offline as tools is **less expensive**, can lead to efficient gameplay improvements for solving complex games without large entry cost.

Related Work

Keywords

Games

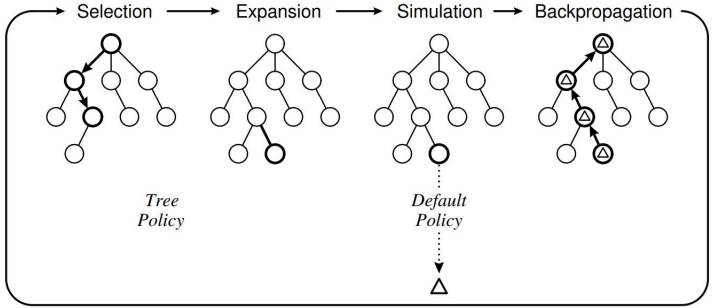
LLM Task Proposal

Tool-creati on Agent System

Monte Carlo Tree Search (MCTS)

A Survey of Monte Carlo Tree Search Methods
[2012] Cameron Browne et. al

- **Challenge:** How to find optimal decisions in given domain (complex game as MDP)?
- **Paper:** Maps problem to search tree, perform 4 stages (selection, exploration, simulation, back propagation) with tree policy to understand game states and identify optimal decisions
- Exploration heuristics like UCT balances exploration and exploitation with simple data
- **Limitation:** Lacks reasoning over environment data, constraint by limited information
- **Our learning:** Solve complex games base on a MCTS framework
- **Our approach:** Use LLM to enhance reasoning over environment and state information



UCT only knows reward and visit counts

$$\textbf{return} \quad \underset{v' \in \text{children of } v}{\arg\max} \frac{Q(v')}{N(v')} + c\sqrt{\frac{2 \ln N(v)}{N(v')}}$$

References

- C. B. Browne et al., "A Survey of Monte Carlo Tree Search Methods," in IEEE Transactions on Computational Intelligence and AI in Games, vol. 4, no. 1, pp. 1-43, March 2012, doi: 10.1109/TCIAIG.2012.2186810. keywords: {Games;Monte Carlo methods;Artificial intelligence;Game theory;Computers;Markov processes;Decision theory;Artificial intelligence (AI);bandit-based methods;computer Go;game search;Monte Carlo tree search (MCTS);upper confidence bounds (UCB);upper confidence bounds for trees (UCT)},

2. MCTS with LLM: Policy-Guided Tree Search (PGTS)

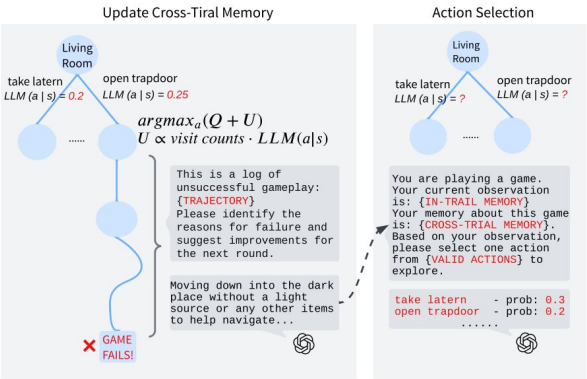
MCTS, LLM

- [AI generated, need revision] Proposes a learned tree policy that dynamically decides whether to expand, branch, backtrack or terminate the search during LLM reasoning. By learning this policy via reinforcement learning, PGTS removes the need for hand-crafted heuristics and reduces computation compared with standard tree search
- **Significance: LLM reasoning has been used to improve tree policy and proven useful, but requires human effort, and lacks capability to address hard problems**

MC-DML (Monte-Carlo planning with Dynamic Memory-guided Large language model)

[ICLR 2025] Zijing Shi et. al

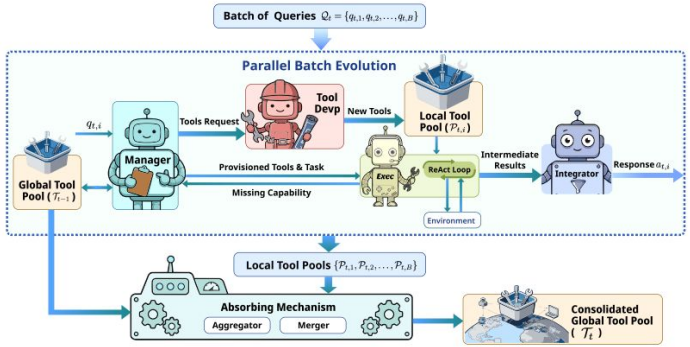
- **Challenge:** Past MCTS+RL limitations, like uncertain exploration and lack of reasoning
- **Paper:** Deploy LLM as policy (action decision) maker in text games, enrich context with dynamic memory mechanism (in-track, cross-track memory)
- **Limitation:** High cost, confusion with local memory management
- **Our learning:** Utilize LLM's reasoning capabilities and memory management
- **Our approach:** We generate heuristics, used for many steps; we study summarized traces in a global point of view, tool generation



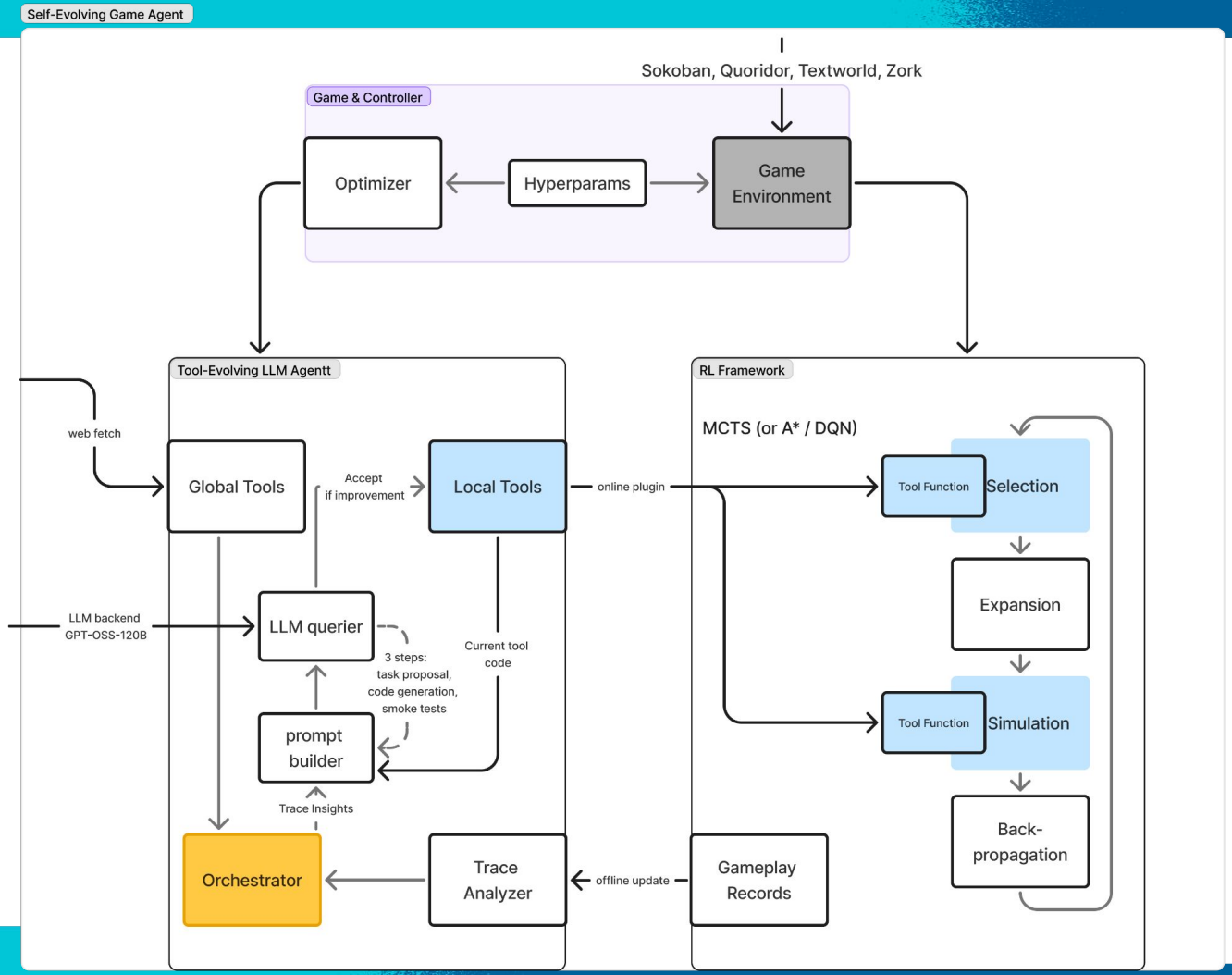
Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks

[2026] Haotian Li et. al

- **Challenge:** LLM requires human supervision to solve hard problems
- **Paper:** Multi-agent architecture that solves sequence of real-world problems by generating, synthesizing, optimizing, and reusing tools (python scripts) in parallel
- **Limitation:** Focuses on deterministic problems, can benefit from targeted workflow
- **Our learning:** Novel tool generation paradigm reduces supervision in our system
- **Our approach:** Using LLM to analyze, generate, and improve MCTS or other heuristics as tools for validate agentic propose task



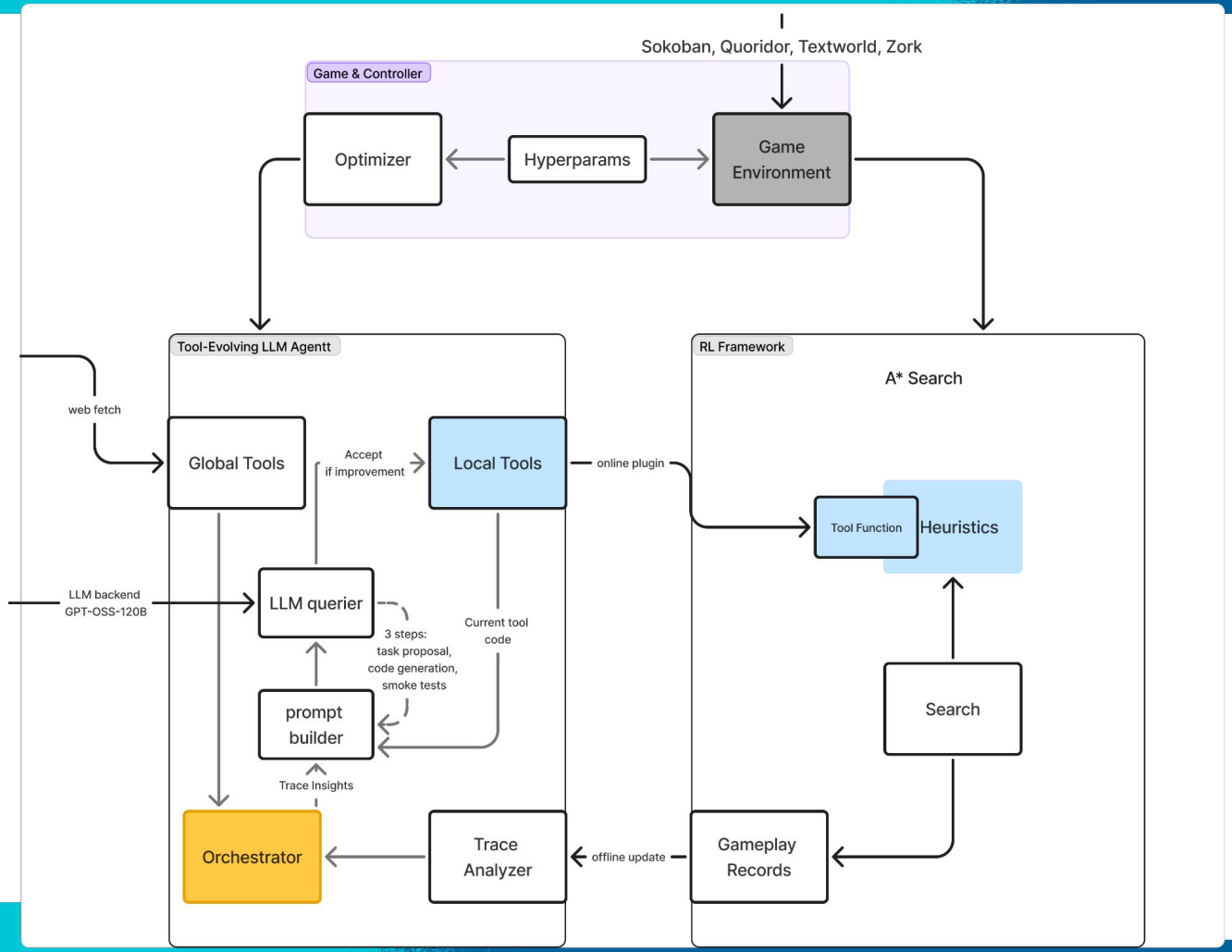
Architecture



UC San Diego

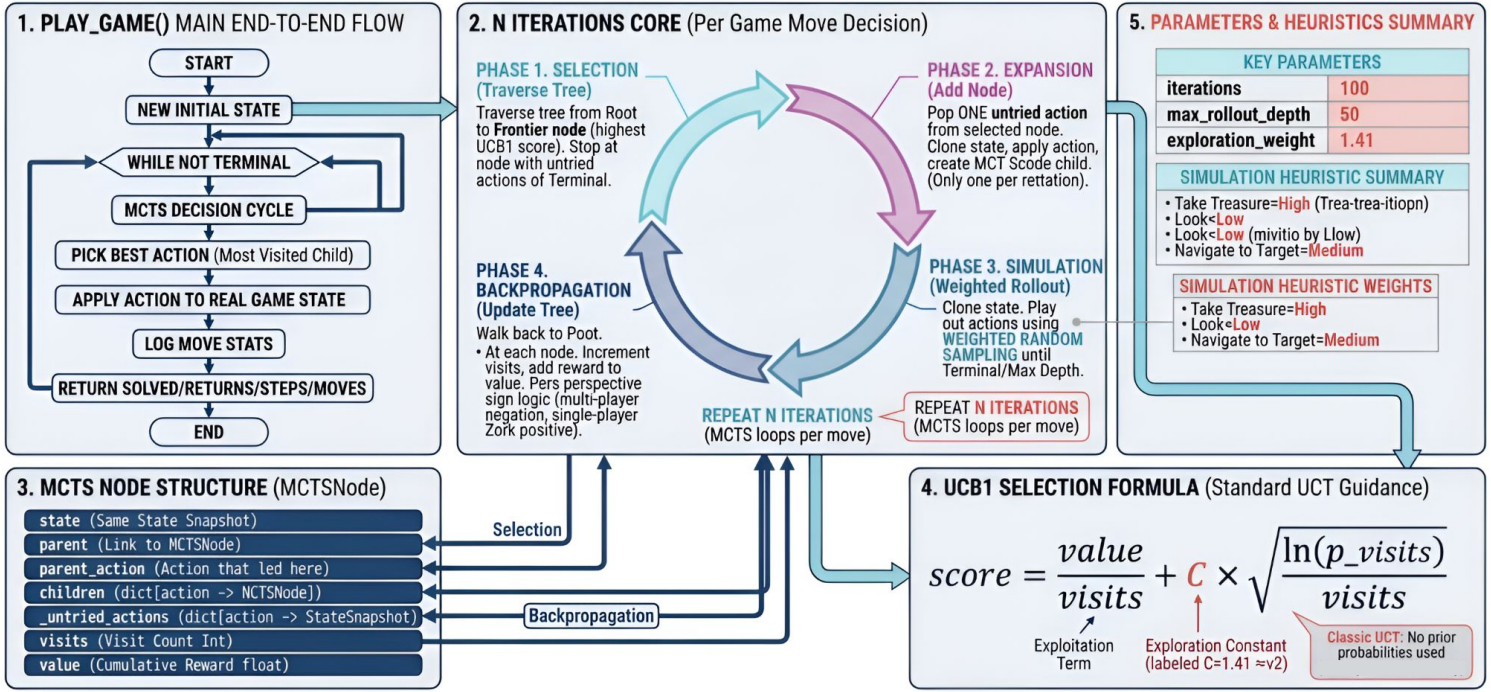
- RL Framework not limited to MCTS
- Modualize optimizable sections, allowing LLM creation & modification
- Adaptable to all turn-based games
- Codebase: https://github.com/zhuyuezx/CSE291A_Project

Self-Evolving Game Agent(A*)

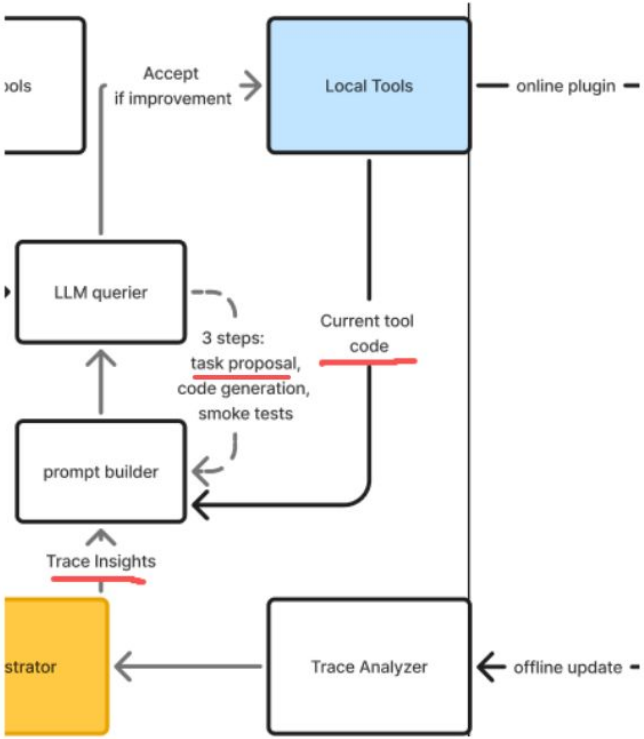


- A* suitable for games with simple action space and easy-to-get heuristics but complex, irreversible state space (e.g. Sokoban)
- MCTS suitable for games with complex action space and hard-to-get heuristics (e.g. go, quoridor)

Introduction



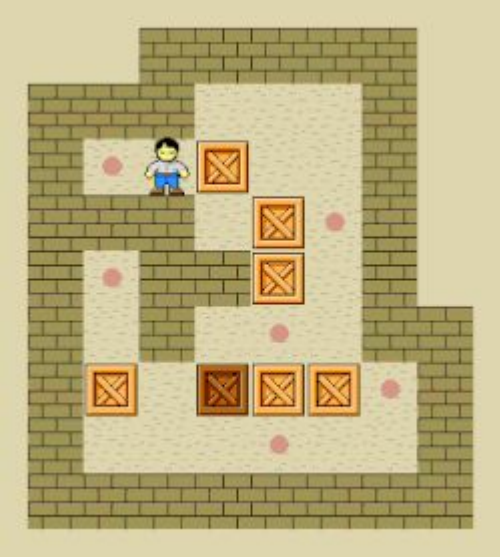
Task Proposal Loop



- Input : Current tool code + Environment feedback given current tools.
- **Mutable source** for improvement seed
- Improvement ideas generated **iteratively**
- Verified and **tested** before incorporated
- Task proposal quality is the **most important fault point**.

Experiments

Sokoban (A* optimization)



```
#####
###   #
#.@$  #
# # $.#
#.# $ #
# # . #
#$ *$$ .#
#   .   #
#########
```

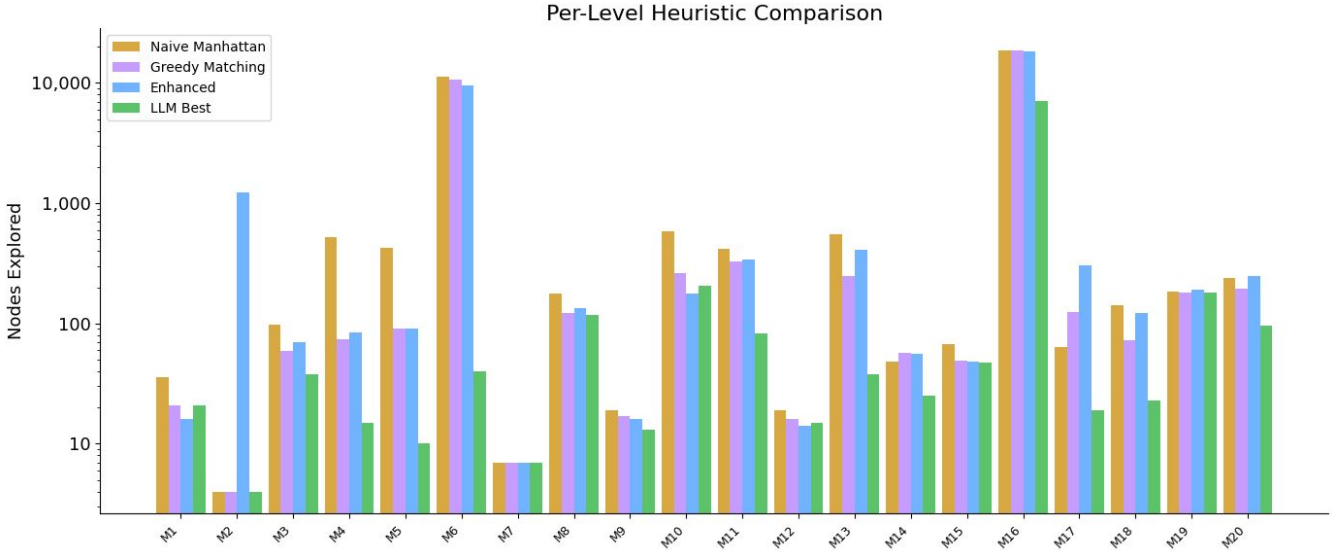
Heuristic Example: **Naive Manhattan Distance**

```
def naive_manhattan(board, boxes):
    """Sum of Manhattan distances"""
    total = 0
    for bx in boxes:
        total += min(bx.manhattan(g) for g in board.goals)
    return total
```

Training on 20 Sokoban Levels

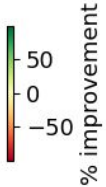
Baseline: Greedy matching + mobility penalty

Hyperparams: 10 iterations, maximum 30,000 search steps

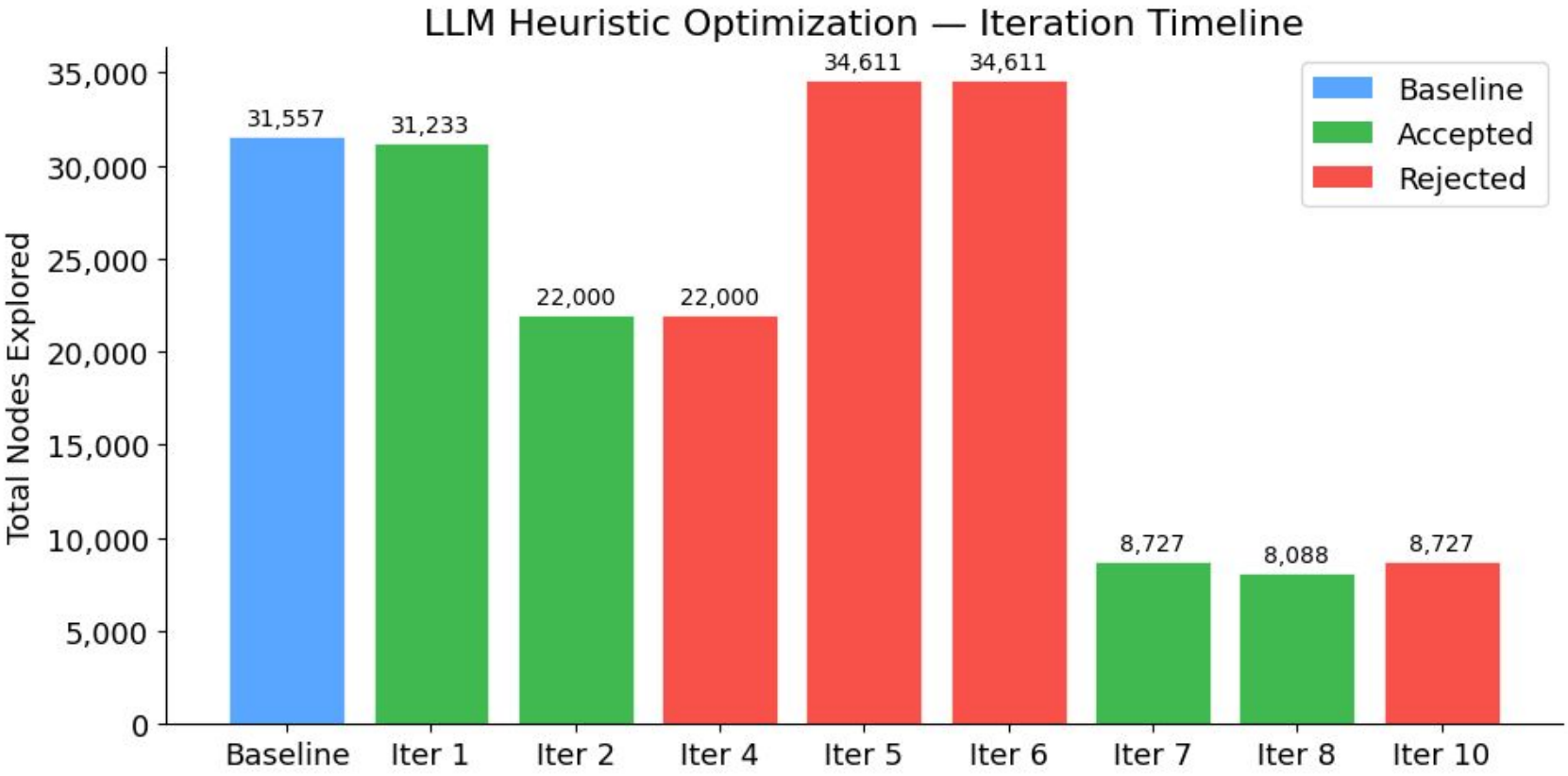


Node Reduction: LLM Best vs Enhanced Baseline (%)

M1 -31%	M2 +100%	M3 +46%	M4 +82%	M5 +89%	M6 +100%	M7 +0%	M8 +12%	M9 +19%	M10 -15%
M11 +76%	M12 -7%	M13 +91%	M14 +55%	M15 +2%	M16 +62%	M17 +94%	M18 +81%	M19 +6%	M20 +62%



Training on 20 Sokoban Levels



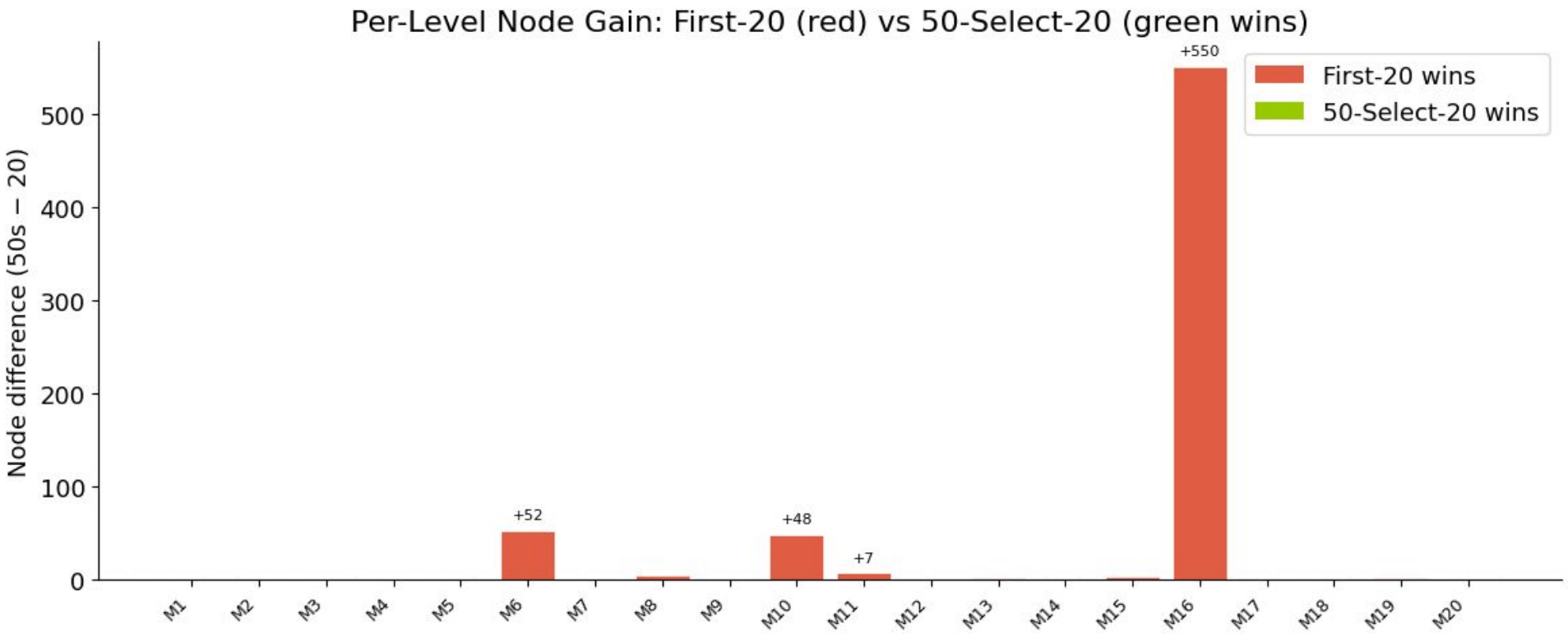
Microban 8	177	122	134	118	↓11.9%
Microban 9	19	17	16	13	↓18.8%
Microban 10	586	263	178	205	↑15.2%
Microban 11	418	325	342	82	↓76.0%
Microban 12	19	16	14	15	↑7.1%
Microban 13	553	247	411	38	↓90.8%
Microban 14	48	57	56	25	↓55.4%
Microban 15	67	49	48	47	↓2.1%
Microban 16	18,618	18,656	18,438	7,091	↓61.5%
Microban 17	64	124	304	19	↓93.8%
Microban 18	142	72	122	23	↓81.1%
Microban 19	185	182	192	181	↓5.7%
Microban 20	241	196	250	96	↓61.6%
TOTAL	**33,479**	**31,268**	**31,557**	**8,088**	**↓74.4%**

Heuristic Source Code

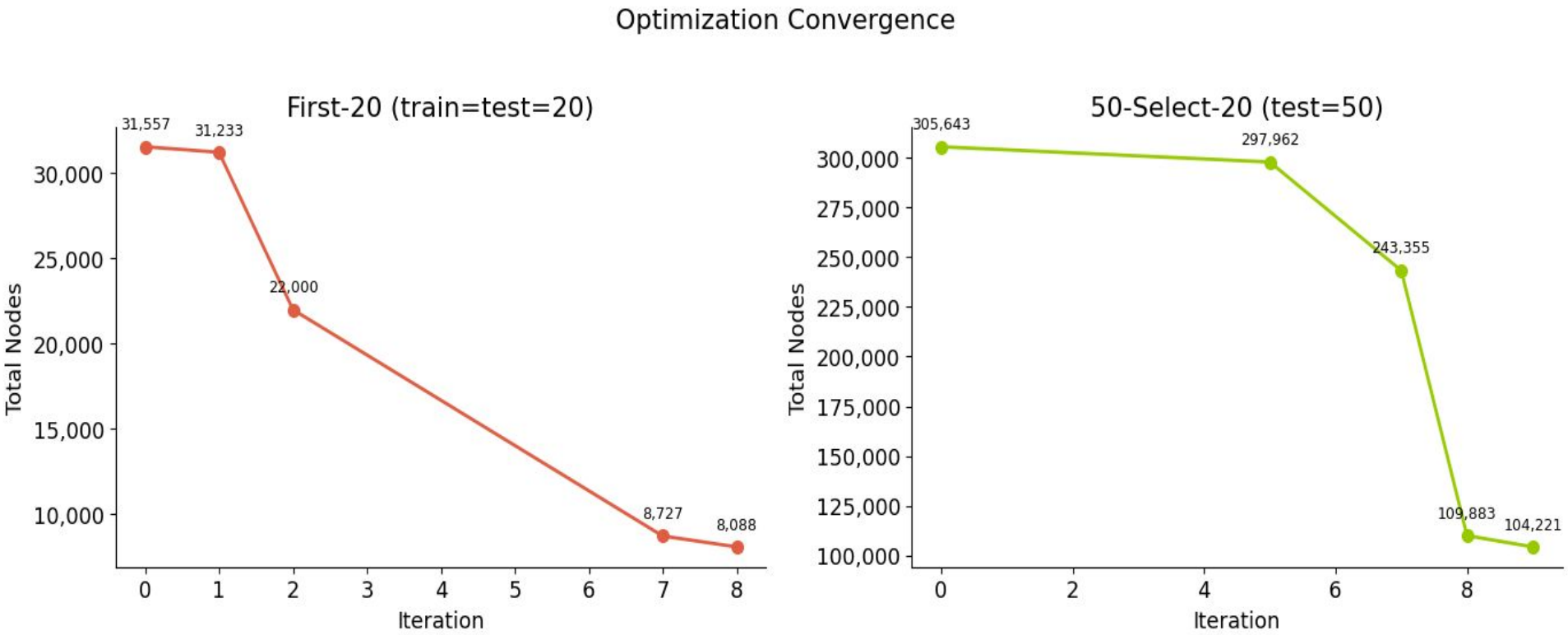
Naive Manhattan | Greedy Matching | Enhanced | llm_iter_1 ✓ | llm_iter_2 ✓ | llm_iter_4 ✗ | llm_iter_5 ✗ | llm_iter_6 ✗ | llm_iter_7 ✓ | llm_iter_8 ✓ | llm_iter_10 ✗

```
def naive_manhattan(board, boxes):
    total = 0
    for bx in boxes:
        total += min(bx.manhattan(g) for g in board.goals)
    return total
```

Interesting: 20 VS 50-select-20

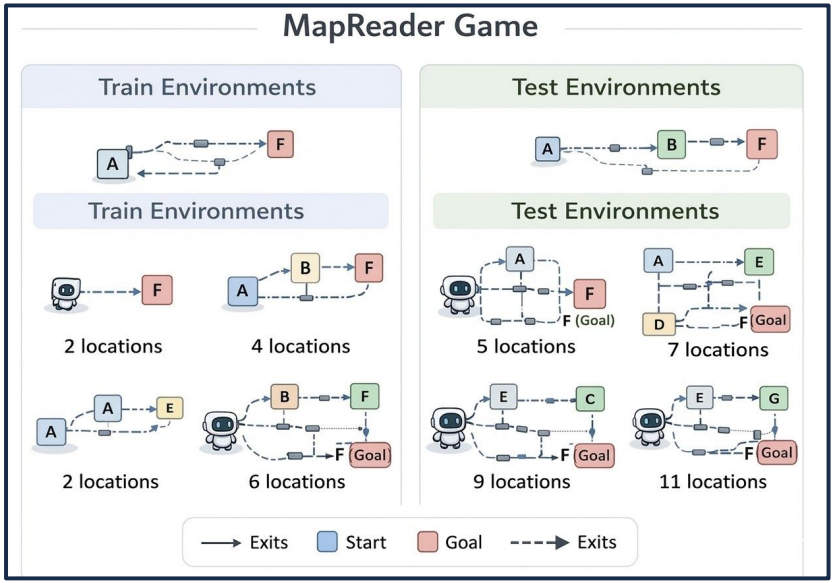


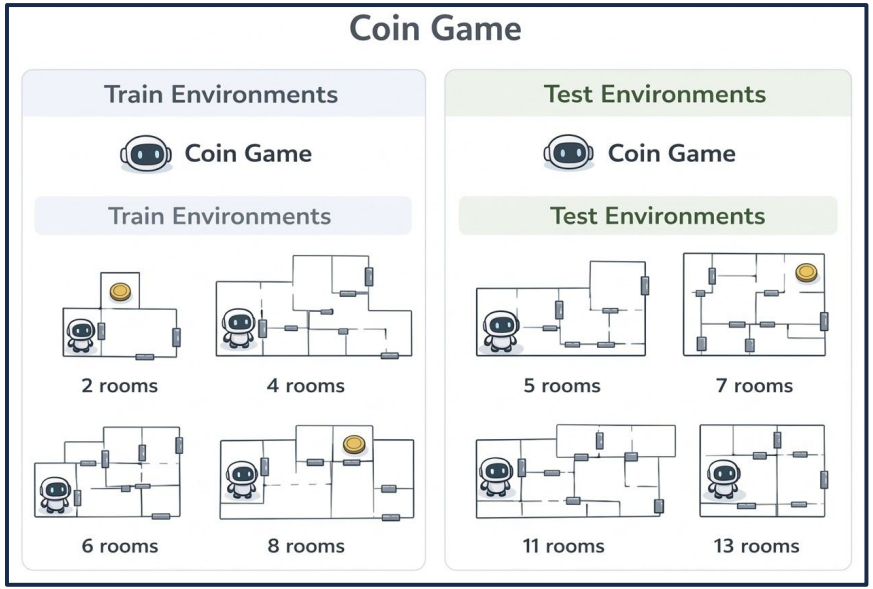
Interesting: 20 VS 50-select-20



Experiment Setup: TextWorld Benchmark

1. Introduce the two games: **Coin** and **MapReader**
2. Use **train environments** to collect gameplay traces
3. Select the **3 hardest/failure cases** for LLM heuristic optimization
4. Evaluate on **larger, unseen test environments**



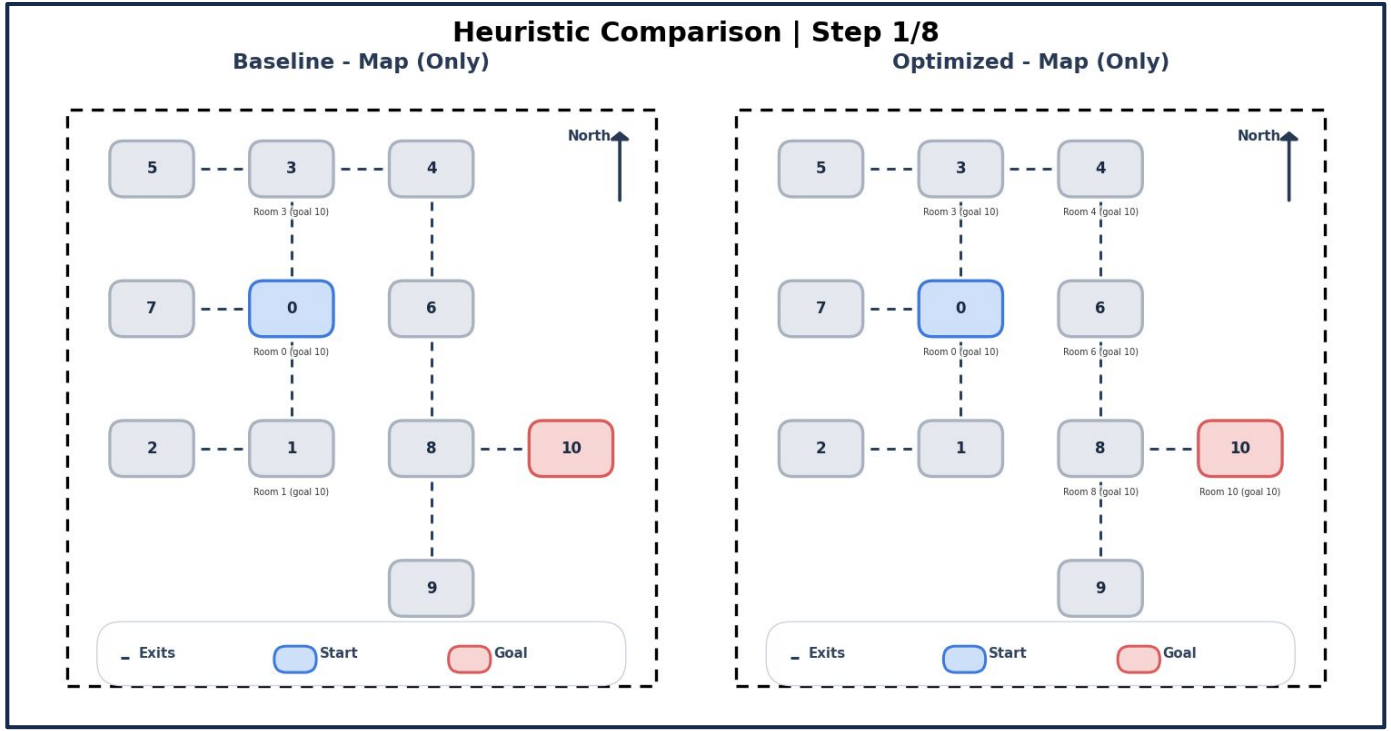


TextWorld Benchmark: Behavior Example

1. Baseline heuristic v.s LLM optimized heuristics for **4 phase**.

2. Base heuristic gets **stuck** in **local** loops

3. Optimized heuristic encourages **global** exploration



TextWorld Benchmark: Optimization Results

UC San Diego

Heuristic difference: **Simulation**

- Baseline: **uniform** random rollout
- Optimized: score-guided rollout
- Rewards task actions and goal progress

Simulation optimization improves test correctness -> generalized heuristic

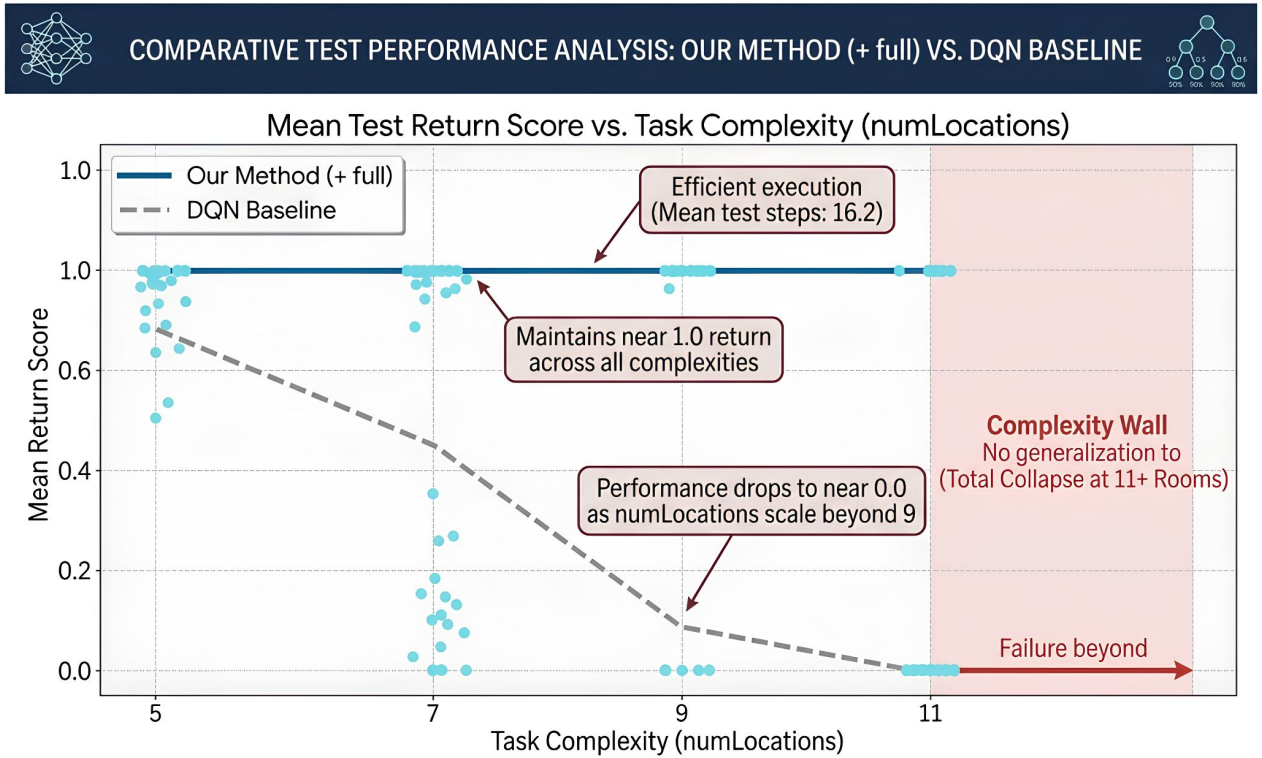
```
# Baseline Simulation

while not terminal:
    a ← random(legal_actions)
    state ← apply(state, a)
```

```
# Optimized Simulation

while not terminal:
    for a in legal_actions:
        score(a) ← heuristic_score(a, state)

    a ← random(argmax score(a))
    state ← apply(state, a)
```

Table 1: Results across configurations on train/test splits. Correct is #correct/#total; Δ is the difference in Correct vs. the baseline on the same split; Ret and Steps are the mean return and mean steps.

Configuration	Train Correct	Train Δ	Train Ret	Train Steps	Test Correct	Test Δ	Test Ret	Test Steps
baseline	62/72	0	0.861	9.9	55/90	0	0.606	14.4
+ selection	63/72	1	0.875	12.8	75/90	20	0.833	15.8
+ expansion	62/72	0	0.819	10.9	55/90	0	0.478	11.3
+ simulation	**72/72**	**10**	**1.000**	41.0	**90/90**	**35**	**1.000**	45.1
+ backpropagation	64/72	2	0.889	**8.5**	60/90	5	0.656	**10.2**
+ full	70/72	8	0.972	14.9	89/90	34	0.989	16.2

Experiment - heuristic generalizability



llm-MCTS pass:
60~90(full)/90

DQN pass rate:
27~33(opt)/90
(test blind - not trained on test env)

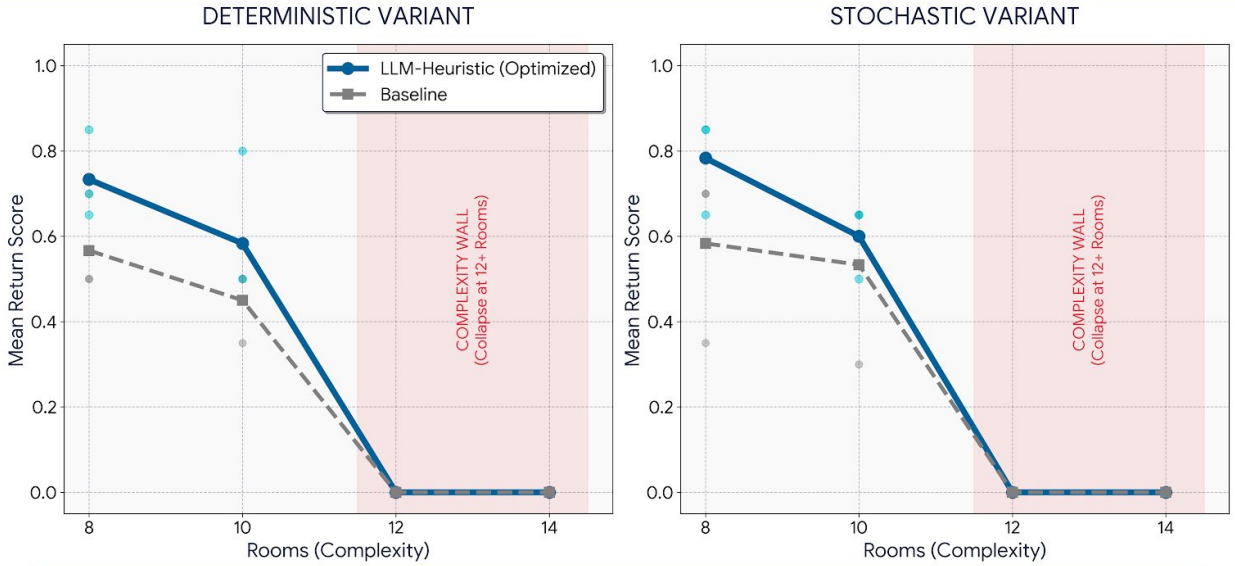
w/:
boltzmann exploration
success replay buffer
stagnation threshold
early stopping
step cost penalty
……

Experiment - with Zork

- Up to 22 interconnected rooms (vs simple grid), with surface + underground layers
- 6 treasures to collect AND deposit back at base (vs pick up 1 coin)
- Multi-step dependencies:
  get lantern → light it → go underground → kill troll → access treasures → return to deposit
- Instant death states: entering dark rooms without lantern
- Resource management: limited lantern fuel
- State space orders of magnitude larger
  (room × inventory × items × fuel × troll × lantern × trophy case)

Experiment - with Zork

PERFORMANCE ANALYSIS: LLM-HEURISTIC MCTS VS. BASELINE



KEY TAKEAWAY: The Optimized MCTS algorithm provides a ~20% performance lift in solvable environments (8-10 rooms), but is limited by the "Complexity Wall" starting at 12 rooms where state-space dependencies peak.

Limitations

- We were not able to meaningfully evaluate **long-horizon behavior** of tool quality
  - After ~1000 code iterations, does the tool quality collapse, plateau, or see eureka moments of continuous improvement?
- Iterative task proposal can get **stuck in parameter tuning** without proposing new ideas. Task proposal could use more research.
- MCTS heuristic tools show some **transferability** on same game different setups
  - Lacks investigation on cross game transferrable tools
- Evaluation limited to **simple environments**
  - Agent demonstrated improvements on multiple simple games, lacked a larger, more impressive game environment to evaluate high impact improvements.

Future Work

- **Border Agent Scope**
  - We limited LLM agent's scope of read/write access to only specific tool files to specific RL frameworks (MCTS) for **feasible evaluation**.
  - The LLM agent might perform better with **elevated scope** - access to full tool folders, freedom to switch RL frameworks that interact with environment, or freedom to modify its own prompt engine/LLM workflow .
- **Task proposal** effectiveness.
- **Generalizing** beyond white-box games
  - The game environments we evaluated are **whitebox** with explicit code endpoint for interaction.
  - Potentially allow LLM agent to figure out/implement interaction frameworks with **blackbox games**. Such as using screenshots to learn game interaction but implement tools to later build encapsulated movesets with mouse clicks.

References

- C. B. Browne et al., "A Survey of Monte Carlo Tree Search Methods," in IEEE Transactions on Computational Intelligence and AI in Games, vol. 4, no. 1, pp. 1-43, March 2012, doi: 10.1109/TCIAIG.2012.2186810. keywords: {Games;Monte Carlo methods;Artificial intelligence;Game theory;Computers;Markov processes;Decision theory;Artificial intelligence (AI);bandit-based methods;computer Go;game search;Monte Carlo tree search (MCTS);upper confidence bounds (UCB);upper confidence bounds for trees (UCT)},
- Shi, Zijing, Meng Fang, and Ling Chen. Monte Carlo Planning with Large Language Model for Text-Based Game Agents. arXiv:2504.16855, 2025.
- Li, Haotian, et al. Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks. arXiv:2601.18226, 2026.

UC San Diego

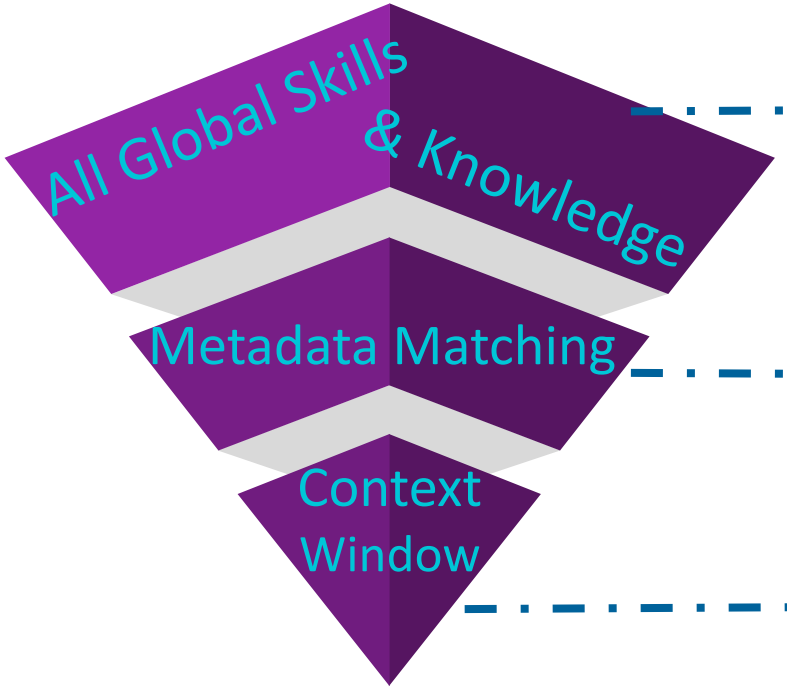
THANK YOU!

Introduction - Project Overview

- Designed as a **generic solution** for any turn-based game.
- A self-evolving hybrid agent : Integrates **Reinforcement Learning (RL)** for core decision-making with a **LLM subsystem** for environmental reasoning.
- Key Capability:
  - **Tool Creation**: The LLM analyzes the game to generate auxiliary tools that the RL agent uses for new actions
  - **Evolving Tool Case:** Rather than fixed rules, the agent constructs a usable interface incrementally with **minimal prior assumptions**

Background

Context vs. Capability: Progressive Disclosure



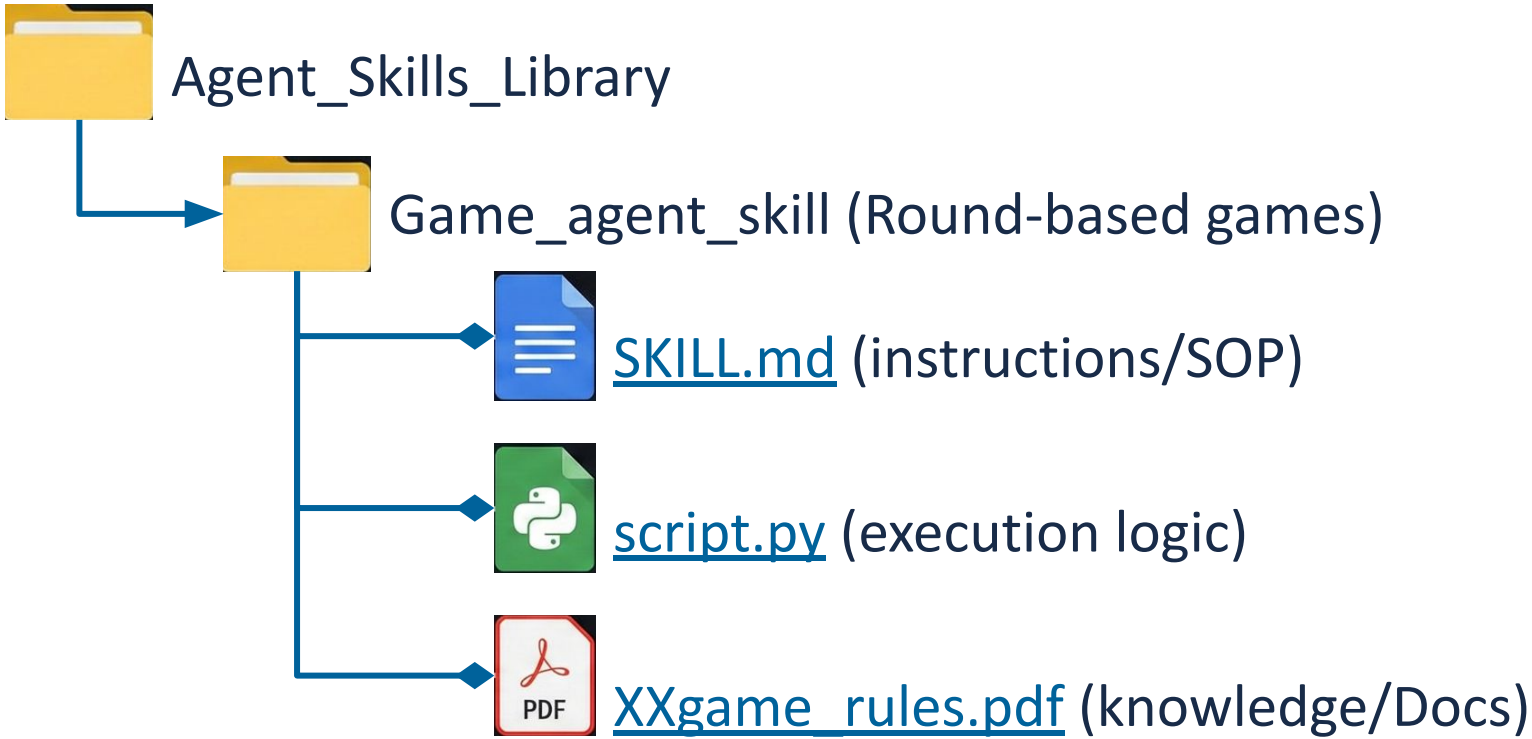
Big Model Capability:
Physics, Coding, History, Medical, etc.
(Big Data, Scaling Laws …)

"Meta Learning":
Expertise, Discipline description ...

Detailed demands:
Loaded for current task.

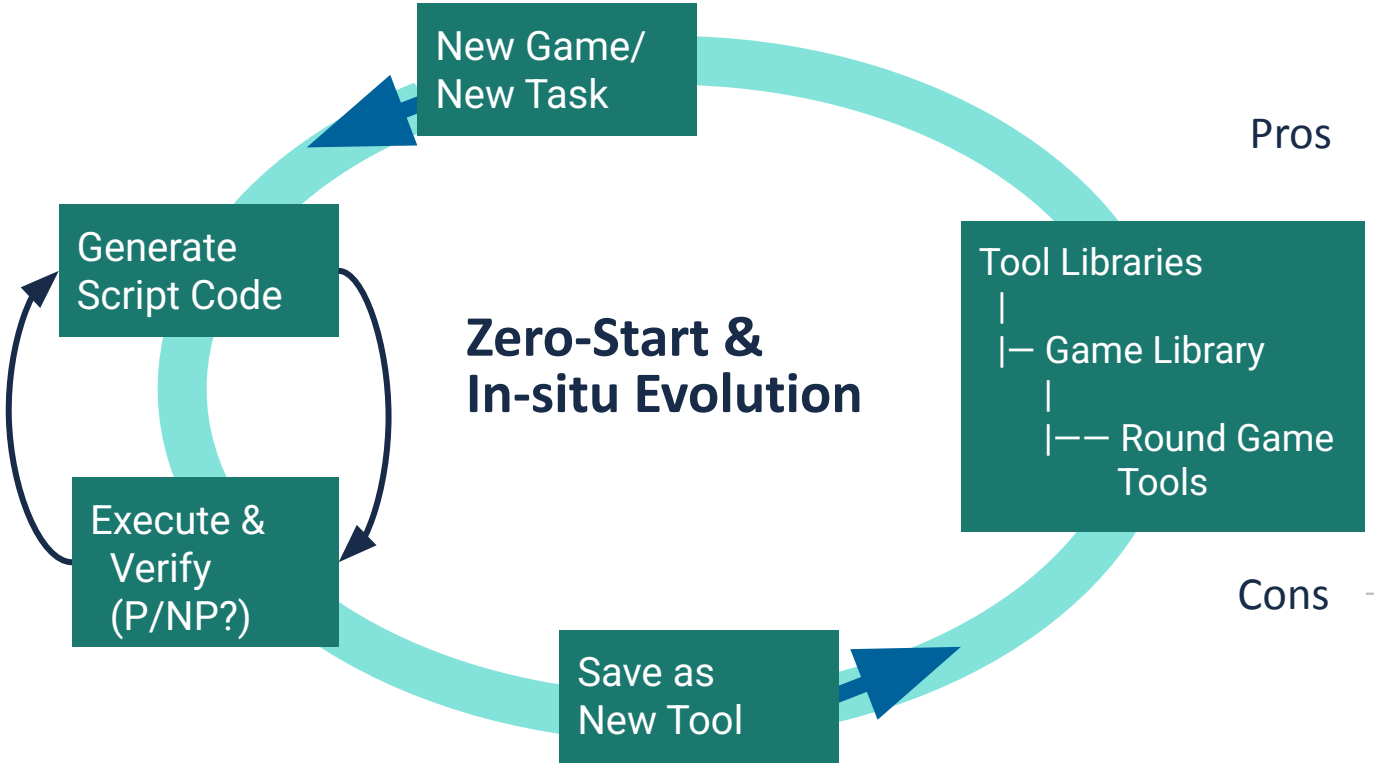
Background

Agent Skills: The Human-Curated Standard



Background

Tool Agent: The Self-Evolving Machine



Zero-Start &
In-situ Evolution

Pros
- Zero-Start Scalability
- Self-Healing
- Infinite Scope

Cons
- Safety Hazard
- Silent Failure
- Depends on binary feedback

Motivation 1:Why Tool-Using LLM Agents?

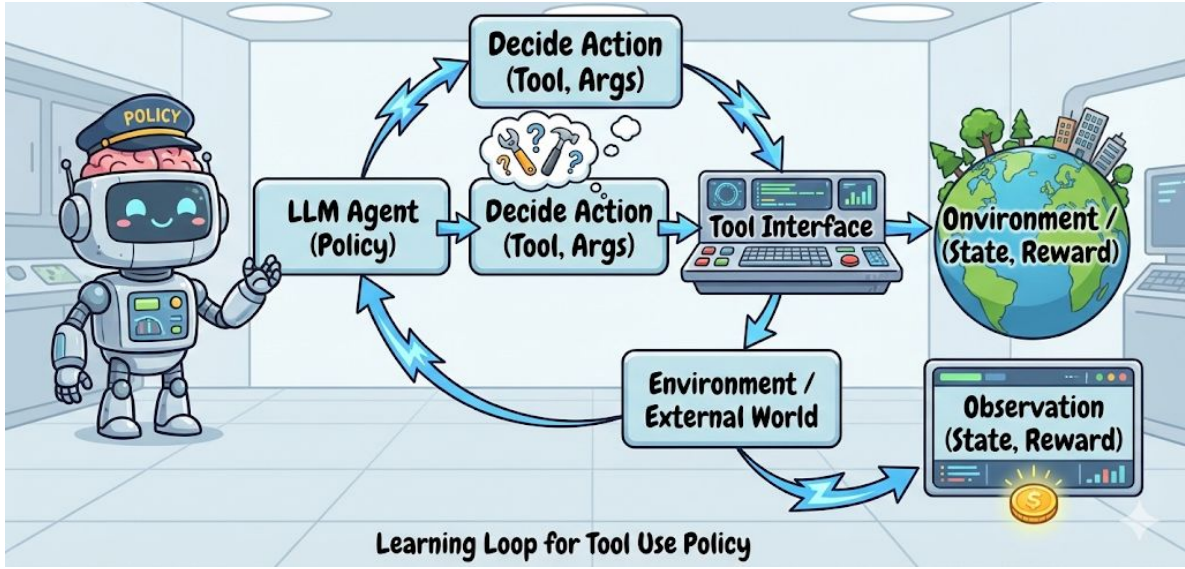
UC San Diego

Current LLM agents use tools via:

- Prompt-based tool invocation
- Hand-crafted rules or heuristics

These approaches are:

- ❌ Expensive (many tool calls)
- ❌ Hard to generalize
- ❌ Environment-specific



Key insight: Tool usage is a ***sequential decision problem*** with long-term rewards.

Motivation: Prompting is insufficient. Tool-use should be learned as a policy.

Motivation 2:Why Games + Yunjue?

UC San Diego

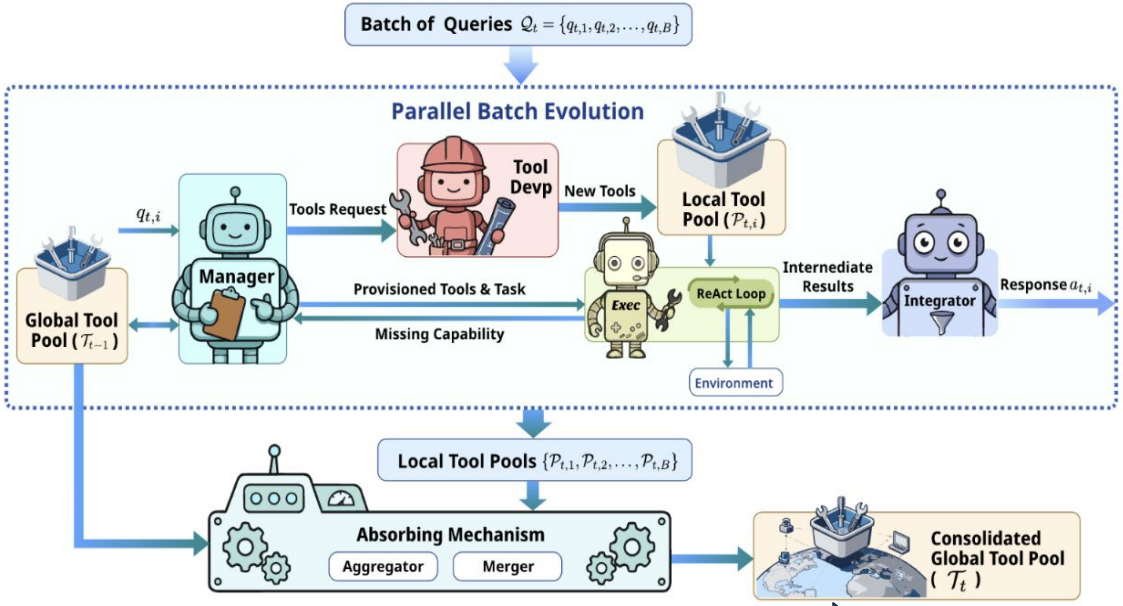
Why Turn-Based Games?

- Clear state–action–reward structure
- Low cost, fully observable
- Rich tool needs: planning, memory, inventory

Turn-based games provide a **clean RL testbed** for studying tool-use.

Why Yunjue?

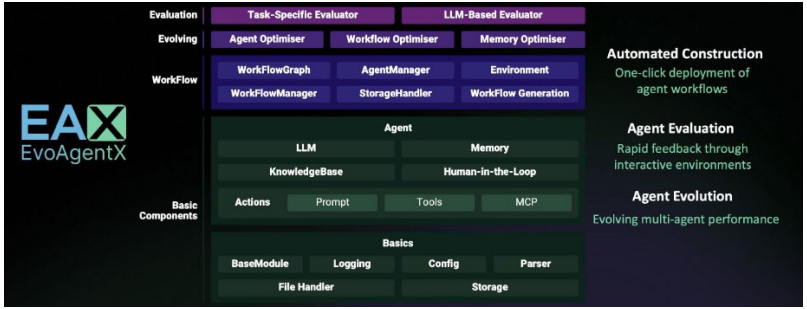
- Provides a general LLM agent interface
- Supports state analysis and tool integration



❗ **But**: Yunjue does not define ***how agents should learn to use tools***.

5. Solve Game play automation with multiagent system: EvoAgentX (2025)

- [AI Generated, need revision]: An open-source platform for automatically generating, executing and evolving LLM-based multi-agent workflows. EvoAgentX uses five layers (basic components, agent, workflow, evolving and evaluation) and integrates optimization algorithms (TextGrad, AFlow and MIPRO) to refine prompts, tool configurations and workflow topologies. Experiments show improved performance on multi-hop reasoning, code generation, mathematics and real-world tasks
- **A raw MCTS system still requires careful human supervision and strict input generation and structuring, a multi agent system can help to decompose simpler input data for new games and establish more automation in game testing process**



Related work

- **Yunjue Agent**
- Article: https://arxiv.org/abs/2601.18226
- Github: https://github.com/YunjueTech/Yunjue-Agent
- An AI agent that can make plan to build skills via python script to solve complex problems, instead of relying on human guidance to plan or build applicable tools
- **Skills**
- https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf?hsLang=en
- Launched by Anthropic on Oct 2025, a skill is a set of instructions that teaches LLM agent to perform a specific task

Related work

- **Yunjue Agent**
- Article: https://arxiv.org/abs/2601.18226
- Github: https://github.com/YunjueTech/Yunjue-Agent
- An AI agent that can make plan to build skills via python script to solve complex problems, instead of relying on human guidance to plan or build applicable tools
- **Skills**
- https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf?hsLang=en
- Launched by Anthropic on Oct 2025, a skill is a set of instructions that teaches LLM agent to perform a specific task

Related work

- **Yunjue Agent**
- Article: https://arxiv.org/abs/2601.18226
- Github: https://github.com/YunjueTech/Yunjue-Agent
- An AI agent that can make plan to build skills via python script to solve complex problems, instead of relying on human guidance to plan or build applicable tools
- **Skills**
- https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf?hsLang=en
- Launched by Anthropic on Oct 2025, a skill is a set of instructions that teaches LLM agent to perform a specific task

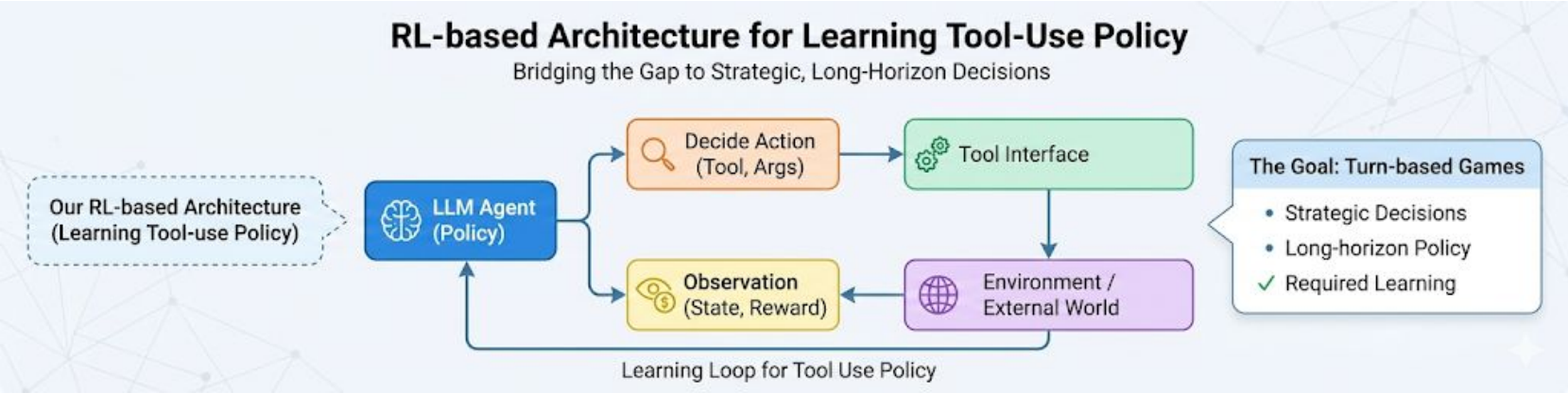
The Research Gap & Our Goal

UC San Diego

The Gap

Feature	Yunjue	Turn-based Games
Tool-use	Abstract API	Strategic Decisions
Decision Type	Manual	Long-horizon Policy
Learning	❌ Not learned	✅ Required

Gap: No mechanism for learning ***when and how*** to use tools in multi-step environments.

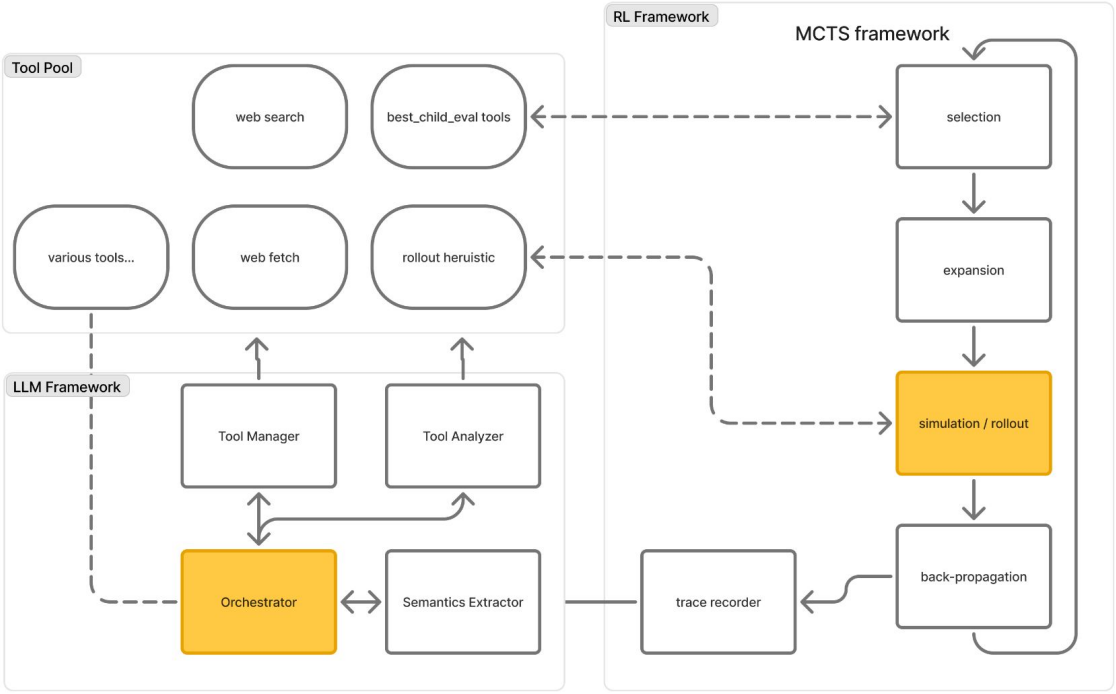


Motivation & Goal: Design a RL-based architecture that enables LLM agents to **analyze, create, and learn to use tools** within Yunjue-based turn-based games.

Impact: Significant exploration of the “skills” concept by attempting to solve large-scale, generalizable problem domain with AI agent

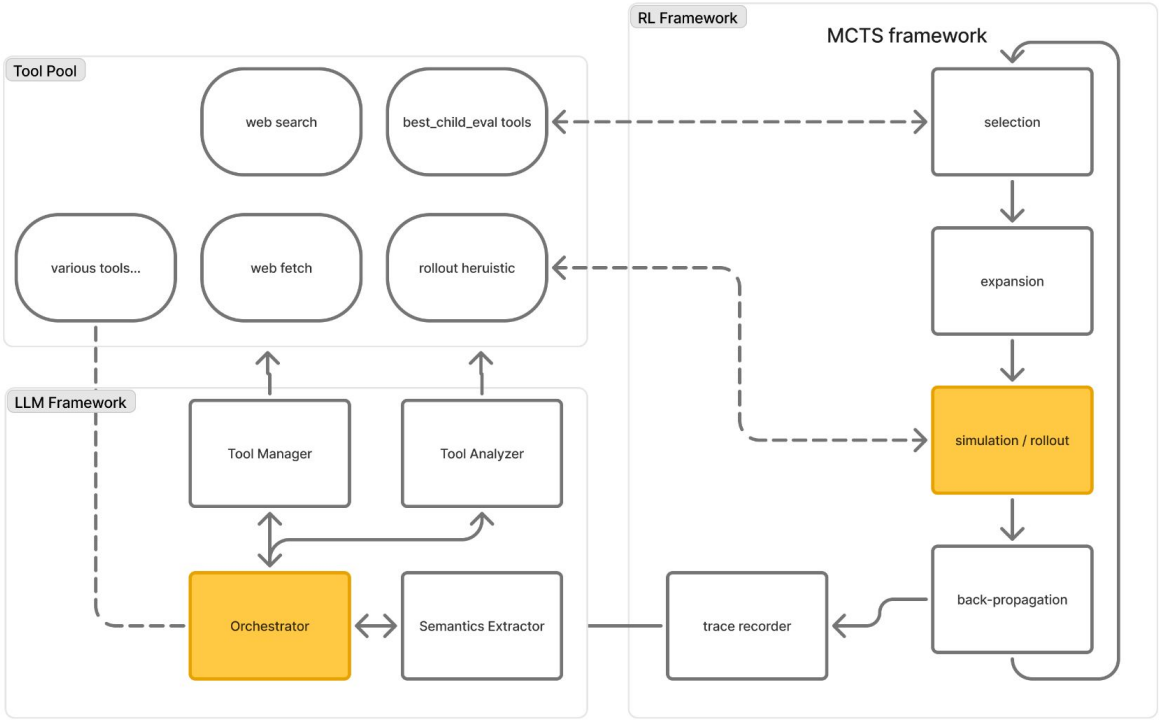
Project Part1: Overall Architecture

Core Designs



- Monte-Carlo Tree Search based RL framework
- Tool use are embedded into RL loop
- Game trace will be recorded and used to optimize tool pool by LLM

CSE291 Architecture



Project Part2: Cross-Game Plan

MCTS Experiment - Quoridor

- Two players take turn, can either place bar or move in four directions.
- Bad performance with naive MCTS, and need manual heuristic fine-tuning
- Number of states similar to Chess, but most actions have no use

Future Evaluation

We wish to demonstrate:

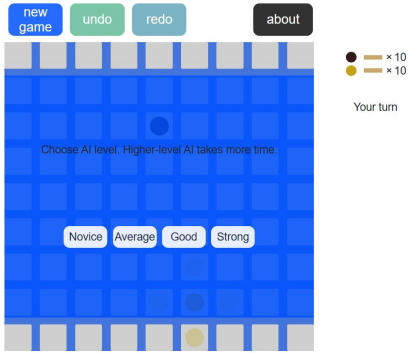
- Superior performance in target game (Ex. Quoridor)
    - Fair measurement based on established, less formal metrics
- Adaptivity to a family of games
    - Eventually, integrated tools should generalize
- Growing performance overtime
    - Better toolbox leads to stronger performance

Case Study Example: Quoridor

- **Initial Research:**
  - Mostly thesis paper, many projects
  - Popular metric: game win rate against other heuristics/agents
- **Our current idea:**
  - Simulate games against existing heuristics or agents, report win rate

p1 \ p2	Random	Semi-Random	A*	Minimax	MCTS
Random		31.3	6.6	5.3	0
Semi-Random	73.5		11.5	10.2	0
A*	92.2	90.2		0	0
Minimax	92.4	88.9	8.2		0
MCTS	90	74.3	100	74	

An Undergraduate Thesis: Artificial Intelligence for Quoridor Game



Level	Rollouts per Move
Novice	2,500
Average	7,500
Good	20,000
Strong	60,000

Online Agent: Quoridor AI with MCTS

Evaluate a Family of Games

- **How to Evaluate Adaptivity?**
  - Start with training model with Quoridor
  - With more tools, train on other games of similar mechanics e.g. Ricochet Robots, Tek (2-player, grid-map, turn-based strategy)
  - Observe usages of established tools and quick improvement of performance
  - Expectation: existing tools help agent to perform better in new games

Table 1: **Comparison between A* and our algorithm.** As the instances get harder the gap between the proportion of solved of A* and our algorithm increases.

Grid Size	#Helpers	#Instances	A*	Our
32 × 32	3	10000	90,8 %	**94,7 %**
96 × 96	3	1000	54 %	**87,8 %**
96 × 96	5	1000	21,2 %	**86 %**

Example: Ricochet Robots Evaluation

1 https://www.scitepress.org/Papers/2023/116999/116999.pdf?utm_source=chatgpt.com

Evaluate Growth Overtime

- **How to Evaluate Impact of New Tools on Performance?**
  - We can evaluate on the same game before and after training all games
- **Current Challenges:**
  - Particular games lack authoritative, high-potential evaluation methods
  - Lacks quantitative evaluation for other advantages
  - Simulations can be expensive
- **Our Next Step:**
  - Continue searching for existing work
  - Explore wider evaluation options like cost/quality tradeoff

Timeline

- **We anticipate the major blockers to be the technical challenges in developing from the new Yunjue agent, establishing outstanding RL architecture, and achieving the generalization and integration of skills.**
- **Week 6: Codebase setup, establish baseline**
  - We expect to settle a workable implementation plan, familiarize major tools like Yunjue Agents, and establish a minimum solution for simple game
- **Week 7: Workable solutions for above major parts**
  - We will complete the implementation of a suitable RL architecture, necessary functionalities based on Yunjue, and prompts or code for optimized tools
- **Week 8: Integration of subparts, optimization and evaluations**
  - We will present an agent that integrates above parts to solve a domain of simpler games through development, optimization, and integration of tools in an efficient way
- **Week 9: Final report and minor fixes**
  - After also complete evaluation in week 8, we will draft the final reports and fill missing details

Cost & Token Usage Prediction: hybrid models

UC San Diego

Phase 1: Implementation & Tuning

Test Games: 50 rounds of testing for a single game

Tokens per Game: ~600k tokens (High estimate, unoptimized).

Cost Calculation (Per 50 Games):

- **Local (Qwen3-7B/14B deployed locally): $0.00**.
- **DeepSeek API (DeepSeek-V3):**
  - *Input:* $0.28 / 1M tokens.
  - *Output:* $0.42 / 1M tokens.
  - *Total Tokens:* 30M tokens (50 games * 600k).
  - *Estimated Cost:* **$9~$12**

Depending on the actual development, we'll spend ~$40 for implementation

Phase 2: Benchmarking & Adaptability Showcasing

Test Games: 3~5 round based games with prior RL cycles

Total RL cycles + Showcasing cycles: ~100 rounds

Tokens per Game: ~600k tokens

Total Volume: 60 Million Tokens.

Cost Calculation (Gemini 3 Flash):

- *Input ($0.5/1M):* 48M * $0.5 = $24
- *Output ($3/1M):* 12M * $3 = $36

Depending on the actual testing, phase 2 total would be $60 ~$80

Cost Prediction: Yunjue case study

UC San Diego

- We will use Yunjue Agent and its embedded GPT-5 model for substantial part of our project, so we will use Yunjue alone to present another sample estimate of our quantitative token expenditure, if we decide to base on Yunjue mostly.
- In previous test case, a prompt involving 4 skills can cost 50k tokens, which is around $0.25 in Openai API platform. Our actual work with simple game should be no less.
- Suppose we be efficient, and allocate 3M tokens ($15) for component development
  - 15 prompts in game prompt development
  - 25 prompts in creating dynamic prompt chain for RL
  - 20 prompts for testing generalizable tools, 10 prompts for testing reward functions
- And we spend 5M ($25) tokens for agent constructions, and 10M ($50) tokens evaluation
  - 100 prompts for integrating components for self-evolving game agent
  - 200 prompts for evaluating agent performance (20 prompts, test set of 10 games)
- So the token cost can be **~18M tokens ($90)** for **GPT-5**. Note that the estimate can differ due to the inaccuracy of initial testing (overestimate token cost) and increased complexity of future development

Cross-game improvement plan

- We plan to continuously improve our agent's tool-kit on a series of games with **increasing complexity.**
- Start with simple games (e.g. connect-4, 2048 etc. from OpenSpiel) to sanity check that tool-use does improve RL performance.
- Migrate the tools onto more complex games while solidifying a toolbox with the tools/heuristics that **transfer** successfully between games.
- Evaluate our agent on games that are costly to perform RL like Quoridor or Chess to show that an effective list of tools can **improve RL performance at low training iterations**.
- With sufficient training iterations, we don't expect to outperform a RL framework like AlphaZero because heuristics are not as flexible. But we believe that our proposed RL agent with LLM synthesized tools/heuristics can perform well on lower training iterations.